



Informationsteknologi

Utbildning

Centrala länkar

Exjobb

Information

Kurser hos oss

Kursstart och registrering

Mixat

Scheman & tentor

Studentorganisationer

Stöd

Utland

Forskning

Forskarutbildning

Om institutionen

Kontakt

Datordrift

Internt

Lösning till alternativ uppgift #1

Sök

Uppgift

Vad gör följande `for`-loop?

```
for(x = 0; x < 100; (v = a = arr + x % 10,
a - arr != 9 &&
*a > *++v &&
(*a -= *v,
*v += *a,
*a = *v - *a),
x++)) ;
```

Svar

`for`-loopen består av fyra delar, initiering, villkor, förändring och kropp. I koden ovan är initiering och villkor uppenbara (`x = 0; x < 100;`) och kroppen är tom, men vad är det som händer i förändringen?

För att bryta upp koden som ligger i förändringen flyttar vi ner den i kroppen. Vi vet att detta går eftersom vi vet att förändringen alltid utförs direkt efter loop-kroppen. Genom att sätta förändringskoden sist i (den tidigare tomma) loop-kroppen får vi:

```
for(x = 0; x < 100; )
(v = a = arr + x % 10,
a - arr != 9 &&
*a > *++v &&
(*a -= *v,
*v += *a,
*a = *v - *a),
x++);
```

Vi ser att kroppen består av ett parentesuttryck med tre satser (`sats, sats, sats`). Eftersom ingen tar hand om returvärdet av parentesuttrycket så kan vi sätta in måsvingar och byta ut detta mot "vanliga" satser:

```
for(x = 0; x < 100; )
{
v = a = arr + x % 10;
a - arr != 9 &&
*a > *++v &&
(*a -= *v,
*v += *a,
*a = *v - *a);
```

```

    x++;
}

```

Enbart för att göra det lite mer läsbart flyttar vi upp den sista satsen till `for`-huvudet igen.

```

for(x = 0; x < 100; x++)
{
    v = a = arr + x % 10;
    a - arr != 9 &&
    *a > *++v &&
    (*a -= *v,
     *v += *a,
     *a = *v - *a);
}

```

Nu ser vi att det i kroppen i själva verket är två satser, den ena är en "enkel" tilldelning och den andra är ett villkorligt uttryck. Vi koncentrerar oss på det villkorliga uttrycket först.

Det är tre satser som binds samman med `&&` (sats `&&` sats `&&` sats). Vi noterar att de första två är logiska uttryck som kommer att resultera i sant eller falskt, och det sista är ännu ett parentesuttryck som innehåller vanlig sekventiell kod. Vi vet att villkorliga uttryck endast evalueras så länge C inte kan avgöra resultatet (dvs (`V1 && V2`) - `V2` kommer endast att evalueras om `V1` är sant, annars vet vi ju redan att uttrycket är falskt). Därför kan vi bryta ut de första två villkoren och sätta dem i en `if`-sats. På samma gång bryter vi ut parentesuttrycket i det sista villkoret till vanliga satser:

```

for(x = 0; x < 100; x++)
{
    v = a = arr + x % 10;

    if (a - arr != 9 && *a > *++v)
    {
        *a -= *v;
        *v += *a;
        *a = *v - *a;
    }
}

```

Nu blev det lite klarare. Vad har då variablerna för typer?

`%` kan endast appliceras på heltal, så `x` måste ju vara `int` (eller motsvarande).

`*` betyder ju att man följer en pekare, dvs hämtar värdet som finns på adressen som finns i variabeln. `a` och `v` bör ju då vara pekare av något slag om allt ska vara rätt.

Eftersom `a` och `v` får sina värden från `arr` så är det väl lämpligt att även `arr` betraktas som en pekare.. Eller varför inte en array? (Pekare och array är ju samma sak och namnet `arr` låter ju lite misstänkt.)

Exakt vilken typ av array och pekare det är går inte att avgöra, men det har å andra sidan ingen betydelse heller. Vi drar till med `int` för att kunna prata om dem.

För att kunna avgöra storleken på arrayen måste vi se vad koden egentligen gör.

`x` kommer att gå från 0 till 99. För varje steg kommer `v` och `a` att peka på en position i arrayen (samma position till att börja med) med index `x % 10`. Indexen kommer alltså alltid att vara 0 till 9 och arrayens storlek kan misstänkas vara 10. Eftersom `x` går från 0 till 99 kommer vi därmed att gå igenom arrayen tio gånger.

Vi har nu:

```
int x;
int *a, *v;
int arr[10];

for(x = 0; x < 100; x++)
{
    v = a = arr + x % 10;

    if (a - arr != 9 && *a > *++v)
    {
        *a -= *v;
        *v += *a;
        *a = *v - *a;
    }
}
```

I `if`-satsen kontrolleras att `a - arr`, dvs array-indexet i `a`, är skilt från 9 (sista positionen) **och** att värdet som `a` pekar på är större än värdet som `v` pekar på *efter* att vi ökat `v` med ett. Vi jämför helt enkelt två efterföljande positioner i arrayen.

Om innehållet på den ena positionen är större än innehållet på den andra positionen utförs koden i `if`-kroppen.

Vad gör då koden i `if`-kroppen??

Om vi förenklar koden något har vi följande:

```
x = x - y
y = y + x
x = y - x
```

Utan att bevisa något säger jag helt enkelt att detta är samma sak som:

```
t = x
x = y
y = t
```

Dvs, vi byter plats på innehållet i x och y fast vi gör det utan en temporär variabel att lagra det ena värdet i. Prova gärna så ser ni att det stämmer.

Nu har vi alltså en loop som går igenom en array med tio positioner tio gånger och varje gång den stöter på två tal där det tidigare är större än det senare så byter den plats på talen.

Det kan ju bara vara en sak: Bubblesort!

Senast uppdaterad: 2005-02-14 19:00:32 Sidansvarig: [Jesper Wilhelmsson](#).

Web: [Kontakt](#)

Copyright © 2007 Uppsala universitet, Institutionen för informationsteknologi.

[Visa utskriftsvänlig sida](#)

[Granska denna sida.](#)

[Redigera denna sida.](#)